

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO HÀNG TUẦN

**Xây dựng Chatbot hỏi đáp về lịch sử Việt Nam sử dụng
GraphRAG**

Lớp: E22HTTT

Tên: Nguyễn Đức Anh

ID: B22DCDT017

Người hướng dẫn: Kim Ngọc Bách

Hà Nội – 2026

Weekly Report 12

Learning Knowledge about Docker and Docker Compose

1. Problem Statement and Solution Overview

In modern software development workflows, especially for web applications integrated with Artificial Intelligence (AI) that require the coordination of complex environmental libraries, a major recurring challenge is the lack of synchronization between the development environment (local machine) and the deployment environment (production server). The phenomenon where software runs stably locally but encounters errors during transfer occurs frequently.

Docker was created to completely resolve this issue through "Containerization" technology. This technology allows packaging the entire source code, libraries, runtime environment, and configuration files into an independent block, ensuring the software can run consistently across any computing platform.

2. Core Concepts of Docker

The Docker ecosystem revolves around three main conceptual pillars:

- **Docker Image:** A read-only template that comes pre-loaded with a minimal operating system, a programming language environment (e.g., Python 3.11), and the application's entire source code. An Image acts as a static blueprint; it cannot be executed directly but is used for sharing and storage.
- **Docker Container:** A living entity created from a Docker Image. When an Image is launched, it becomes a Container. Each Container operates completely isolated from the host operating system and other Containers, allocated with its own network resources and storage space.
- **Dockerfile (Automation Script):** A plain text file containing a sequence of command-line instructions. The Docker Engine reads this file to automatically

download necessary components and assemble them into a complete Docker Image.

3. Basic Structure and Instructions in a Dockerfile

A standard packaging script file typically includes the following foundational instructions:

- **FROM:** Declares the base environment (Base Image) to build the application. This is always the first command.
- **WORKDIR:** Sets the default working directory inside the Container. All subsequent commands will be executed at this path.
- **COPY:** Copies files and directories from the local file system (Host) into the Image's file system.
- **RUN:** Executes shell commands during the Image build process, often used to install dependency packages (e.g., `pip install`).
- **EXPOSE:** Declares the network ports that the Container will listen to when running (e.g., port 8501 for internal web interface applications).
- **CMD:** Provides the default execution command when the Container starts. Only one CMD instruction can take effect per Dockerfile.

4. Docker Compose - The Orchestration Tool

In actual deployment, an IT system often requires multiple services running in parallel (e.g., web application, database, storage service). Manually starting each Container using independent docker run commands makes establishing network communication and managing environmental parameters difficult.

Docker Compose is a tool that helps define and run multi-container Docker applications.

Operating Mechanism: The entire system architecture is centrally declared in a YAML format file (`docker-compose.yml`). This file defines the following aspects:

- Services: The list of containers that make up the application.
- Ports (Port Mapping): Rules for connecting network ports from the Container to the physical host, allowing external access.
- Environment (Environment Variables): Automatically loads security files (like .env) into the container's memory, helping hide sensitive configuration keys from public source code.
- Volumes: Establishes a persistent data storage mechanism, preventing data loss when the Container restarts.

Instead of executing a complex sequence of commands, developers simply need to use the `docker-compose up -d` command for the system to automatically build, set up the network, and run the entire software architecture in the background.